

Table of Contents

1. Executive Summary	1
2. Phase Scope and Phase Ledger	2
3. Discovery Architecture and Search Flow	3
4. Ghana Location Architecture	4
5. Service Matching and Provider Eligibility	5
6. Location Matching and the Ranking Algorithm	6
7. Filters, Sorting and Pagination	8
8. Provider Profiles, Public Discovery and Authentication	10
9. API Endpoints and Database Changes	11
10. SEO, Performance and Security	12
11. Verification Evidence	13
12. Operational Findings, Known Issues and Deferrals	15
13. Git Commits and Phase 5 Recommendation	16

1. Executive Summary

56 Phase 4 verification checks, all passing	283 Total checks across the four suites	21 / 21 Browser E2E steps across three viewports	50 Content-backed URLs in the sitemap
--	--	---	--

Phase 4 of the Dwellers platform turned the product from an authenticated application into a real marketplace. Building strictly on the Phase 1 security foundation, the Phase 2 domain backend and the Phase 3 authentication layer, the phase delivered the nationwide discovery experience: a customer anywhere in Ghana can type what they need, choose where they need it, and see the providers that genuinely serve that location. The work was performed against the 52-part Phase 4 requirements brief supplied by the project owner, and every part of that brief is accounted for in this report. Nothing from Phases 1 to 3 was rebuilt, replaced or duplicated - no second authentication system, no second location or category system, no parallel provider or service-area models.

Three design decisions define the phase. First, discovery has exactly one backend: a single `discoverProviders` service, driven by one central ranking configuration, serves the homepage search panel, the `/find` results page, the category and location landing pages, the provider directory and the public REST API. Search logic exists in no React component. Second, matching is grounded in real database rows, never in free text: a provider appears for a search only if an `ACTIVE` service row links it to the requested category subtree and a `ProviderServiceArea` or `BusinessServiceArea` row covers the requested location. Third, the system is honest by construction: missing information scores neutral instead of fabricated, unverified providers are never called verified, suspended accounts disappear from public view on the very next request, and nearby results are always clearly separated from exact matches.

The proof is behavioural, not cosmetic. The new `verify-phase4` suite adds 56 checks on top of the Phase 1 (48), Phase 2 (106) and Phase 3 (73) suites for a combined battery of 283 checks, all green, including the five critical acceptance tests: the Nsawam plumber appears for Plumbing plus Nsawam; the Accra-only plumber does not; the suspended plumber does not; the Nsawam electrician does not appear in a plumbing search; and thirteen matching providers paginate as ten plus three, never unlimited. A clean-database rebuild was executed again this phase, and a 21-step browser E2E walked the full customer journey at 390, 768 and 1440 pixel widths with zero console errors and zero horizontal overflow. The project now sits at the Phase 5 gate, Request Service / Job Requests, awaiting the project owner's explicit approval.

2. Phase Scope and Phase Ledger

2.1 What the phase covers

Phase 4 spans the whole discovery surface: the homepage WHAT and WHERE search panel, the /find results page with server-side filters, sorting and pagination, the cascading Ghana location selectors, the database-driven category and service taxonomy, provider result cards, the public provider profile, the nearby-provider fallback, shareable search URLs, indexable SEO landing pages, the centralized discovery API, the analytics event foundation, and the documentation set that explains all of it. Every surface reads from the same engine, and every result shown is a real database record - the phase adds no fabricated content anywhere.

Equally important is what the phase deliberately does not build. The job request workflow, quotations, payments and checkout, full messaging, WhatsApp and SMS integration, equipment rental transactions, project management, an AI recommendation engine and a construction cost estimator all belong to later phases. Where the customer journey reaches one of those boundaries, the interface shows an honest, clearly-labelled transition - for example a Request Service button that preserves the customer's intent through authentication and then states plainly that job requests arrive with Phase 5 - rather than faking a submission.

2.2 The starting seam

The phase began with an inspection of the existing seams rather than a rebuild. From Phase 2 it inherited the 41-model domain - in particular ProviderProfile, ProviderServiceArea, BusinessServiceArea, Service, the Category tree and the Region, District, Town and CommunityArea hierarchy with real latitude and longitude columns. From Phase 1 it inherited the API handler factory with its auth, validation, rate-limit and logging pipeline, the pagination contract and the error envelope. From Phase 3 it inherited live account-status enforcement, the service-area pickers that keep provider data truthful, and the design system that keeps the interface visually continuous. The only schema change the phase required was one new analytics table, which is documented in chapter 9.

2.3 Phase ledger

Phase	Scope	Status
1	Foundation, architecture, security primitives, brand	COMPLETE
2	41-model schema, API foundation, seed, location data	COMPLETE
3	Authentication, registration, role dashboards	COMPLETE
4	Find / nationwide discovery (this report)	COMPLETE
5	Request service / job requests / messaging / quotations	AWAITING OWNER APPROVAL

Table 2.1 - Project phase ledger after Phase 4

The remainder of this report maps one-to-one onto the 29 sections required by the Phase 4 specification: chapters 3 to 10 cover architecture, matching, ranking, filters, profiles, API, database, SEO, performance and security; chapter 11 carries the test, browser, device and Nsawam acceptance evidence; chapters 12 and 13 close with findings, deferrals, commits and the Phase 5 recommendation.

3. Discovery Architecture and Search Flow

3.1 One engine, many doors

The central rule of the phase is that discovery has one backend and many entry points. The homepage search panel, the /find results page, the /find/[category] category landings, the /find/[category]/[location] location landings, the provider directory and the GET /api/discovery/providers endpoint all funnel into the same discoverProviders(criteria) service. No page implements its own search query, no component holds matching logic, and no surface can drift from the others, because there is nothing to drift from. When the ranking weights change, every door changes with them.

3.2 The ten-step pipeline

Step	Stage	Behaviour
1	Validate criteria	Strict Zod schema; unknown keys rejected; page and pageSize clamped
2	Resolve location	Community to town, district sweep, region sweep, or explicit coordinates
3	Resolve service or category	ID or slug; subtree expansion to depth 3
4	Hard filters	Active account, offered service, covering service area, public eligibility
5	Fetch candidates	Single capped query (600), minimal selects, no N+1
6	Score relevance	Central weights only; neutral fill for missing data

Step	Stage	Behaviour
7	Rank	Total score descending, stable tie-break by provider id
8	Sort	Requested order applied; null data never falsely ranks
9	Paginate	Server-side slice; meta with total and totalPages
10	Return	Standardized, privacy-safe envelope plus labelled nearby tier

Table 3.1 - discoverProviders pipeline

Hard filters run before any scoring, so a suspended provider or a provider whose service areas never touch the requested town is never fetched at all, let alone ranked. Scoring only orders the providers that already passed, which keeps the ranking honest and the query cheap. The nearby fallback reuses the same hard filters with only the location condition swapped, so a nearby provider is never someone who failed the service or eligibility test.

3.3 Search flow from the browser

A customer journey begins on the homepage, where a WHAT combobox lists the real category tree from the database and a WHERE cascade loads regions, then districts, then towns from the location API - each level fetched only after the previous choice, never one giant dropdown. Submitting navigates to /find with the criteria in the URL: category slug plus location IDs. The /find page is a server component that calls the discovery service directly, renders results, and keeps every filter and sort choice in the URL, so refreshing preserves the search, the back button behaves, and any search can be shared by copying the address bar. The same criteria expressed as an API call return the identical result set, which the verification suite asserts explicitly.

4. Ghana Location Architecture

4.1 The database hierarchy

Discovery is nationwide by design, not Accra-centric. The phase consumes the existing five-level Ghana hierarchy - Ghana, Region, District or Municipality, Town, Community area - exactly as seeded in Phase 2, with all sixteen regions supported and every level carrying real coordinates where they exist. Location identity is internal database IDs end to end; names are display labels only. The frontend hard-codes no region, district, town or service anywhere: the homepage cascade, the filter panels and the SEO landings all read the live tables, so a new town added to the database is immediately searchable.

Level	Rows	Role in discovery
Region	16	Region sweep; cascade level 1
District / Municipality	54	District sweep; cascade level 2
Town	54	Hard location filter; cascade level 3; service-area target
Community area	23	Cascade refinement (optional level 4)

Table 4.1 - Location reference data after seed (verified this phase)

4.2 Cascading selection

The location selectors cascade in four steps - region, district, town, optional community - and each level loads only after its parent is chosen. Choosing Eastern enables the district list for Eastern; choosing Nsawam Municipal enables its towns; choosing Nsawam optionally offers its communities. Defaults are never pre-selected, because a silent default would silently change the customer's search. The same discipline applies on the server: when a district or region sweep runs, the response labels the level honestly, and the requested town context is retained so exact-match providers still outrank wider-area ones.

4.3 Breadcrumbs and shareable context

Every results page renders a breadcrumb built from the resolved hierarchy - for example Ghana, Eastern, Nsawam Municipal, Nsawam - with links back to the region and town searches. The breadcrumb is not decoration: it is the same resolved location object the engine used, so what the customer sees, what the URL says and what the engine filtered on are always the same thing. This is also what keeps SEO landings honest, as chapter 10 describes.

5. Service Matching and Provider Eligibility

5.1 What offers the service means

A provider offers a service only if an ACTIVE, non-deleted Service row links the provider to a category inside the requested subtree. Searching Plumbing expands to the Plumbing category and all of its descendants; searching a parent such as Construction Services finds the specialists beneath it, and those ancestor hits score below exact category hits so the specialist still outranks the generalist. Service names are never matched against free text, and the seed taxonomy - 41 categories from Construction Services down to individual trades - is the only taxonomy in the system.

5.2 Hard eligibility filters

Filter	Rule
Account status	LIVE check that the owning user is ACTIVE; suspension hides a provider mid-session
Public eligibility	Profile not deleted; suspended or deactivated accounts never render as public results
Service match	At least one ACTIVE Service row in the requested category subtree
Service area	A ProviderServiceArea or BusinessServiceArea row covers the requested location tier
Verification filter	Optional VERIFIED, PENDING or UNVERIFIED constraint
Availability filter	Optional AVAILABLE, BUSY or UNAVAILABLE constraint on the real field
Provider type	Optional INDIVIDUAL (no business) or BUSINESS (business-affiliated)
Price bound	Optional maximum starting price in integer pesewas

Table 5.1 - Conditions applied before any ranking

The suspension filter deserves emphasis because it is a live read, not a cached flag: the engine requires the owning user row to be ACTIVE at query time, so an account suspended between two searches disappears from the second one, and its public profile returns 404 immediately. This is the Phase 3 live-status principle applied to public discovery, and acceptance test 4 depends on it.

5.3 Candidates, not unlimited rows

Eligible candidates are fetched in a single capped query - at most 600 provider profiles, selected to the minimal field set the scorer needs, with services and service areas joined through Prisma's filtered relation loads. There is no N+1 anywhere in the hot path, and pagination happens after ranking inside the cap. The cap is a documented, central constant, not a silent truncation: with real Ghana-scale data the correct long-term answer is PostgreSQL-side ranked pagination, which is noted in the upgrade path in DISCOVERY.md. One filter that SQL on SQLite cannot express cleanly - minimum rating with a minimum review count - is applied immediately after fetch, before scoring.

6. Location Matching and the Ranking Algorithm

6.1 Exact location first

Location matching has one golden rule: a provider who explicitly serves Nsawam receives the strongest possible match for a Nsawam search. Match quality descends through the hierarchy - exact community or town match scores 1.0, a district sweep 0.6, a region sweep 0.3 - and when the requested town is known, any provider whose primary location or service-area rows contain that town gets the exact-match score even under a wider sweep. A provider who serves only Accra is not reinterpreted as serving Nsawam:

they either match a wider tier honestly or they are absent, and acceptance test 2 pins this behaviour.

6.2 Distance where coordinates exist

When the requested location and the provider's towns carry real coordinates, the engine computes Haversine great-circle distance to the nearest matching town and maps it onto coarse bands. Bands are deliberately coarse because they exist to rank and to explain, and because the provider's exact coordinates never leave the service layer - the result carries a band label such as 0 to 5 km, never a raw coordinate pair. Where coordinates are missing the engine falls back to the hierarchy scores rather than inventing a distance.

Band	Score	Use
0 - 5 km	1.00	Effectively local
5 - 15 km	0.80	Nearby town
15 - 30 km	0.55	Same corridor
30+ km	0.35	Wider area

Table 6.1 - Distance bands (central configuration)

6.3 The weight matrix

Component	Weight	Data source	Missing-data behaviour
Service match	30%	ACTIVE Service rows	Neutral 0.5 when no category searched
Location match	25%	Service-area rows, hierarchy, distance	Exact tiers and bands as above
Availability	15%	Real availabilityStatus field	UNKNOWN maps to 0.5, never inferred
Rating / reviews	10%	PUBLISHED review aggregate	Neutral 0.5 below 3 reviews
Response rate	10%	responseRatePercent when present	Neutral 0.5 when null
Experience	5%	yearsExperience, full at 10 years	Neutral 0.5 when zero
Verification	5%	Platform verification status	Unverified scores 0

Table 6.2 - Recommendation weights (sum asserted to 1.0 at module load)

The weights live in one configuration module and are asserted to sum to one at load time, so no component can scatter a private weight. The verification suite asserts both the sum and the exact product matrix. The score itself is internal: the API and the interface expose plain-language reasons - Serves your area, Verified provider, Highly rated, Available now, About 5 to 15 km away - and never the number, and the word best is never used. This is ranking transparency without leaking the machinery.

6.4 Neutral fill: the bug the tests caught

The verification suite exposed a real scoring flaw during the build: an earlier draft renormalised the weights over the components that happened to have data, which let an empty profile - no rating, no response rate, no availability - score a perfect hundred percent because its missing components were simply excluded from the denominator. The fix is neutral fill: a component without real data scores 0.5 and keeps its weight, so missing information pulls a provider toward the middle of the ranking instead of manufacturing a perfect one. Unrated professionals sit below genuinely highly rated ones and above poorly rated ones, and no profile can ride a thin denominator to the top. The fix was made in the engine, not the tests, and the regression is now pinned by the recommended-ranking checks.

6.5 Nearby providers as a labelled tier

When nothing matches exactly and the search has a real point to measure from, the engine widens the location condition to towns within a 50 kilometre radius while keeping every other hard filter, ranks those providers by nearest served town, and returns them in a separate nearby block that never merges into the exact results. The interface labels the block Nearby providers and states plainly that these providers serve nearby towns, not the selected location - the customer's choice is never silently replaced. The browser pass verified this live: searching Electrical in Ejisu, where nobody serves exactly, surfaced the Kumasi electrician about 29 kilometres away under the nearby heading.

7. Filters, Sorting and Pagination

7.1 Server-side filters

Every filter operates on the server. The browser sends parameters; the engine applies them inside the query or immediately after the capped fetch; the browser never filters a result list it has already received. Each active filter also stays in the URL, so a filtered view is a shareable view, and the filters panel collapses into a mobile sheet at small widths without changing the contract.

Filter	Parameter	Notes
Service / category	category, serviceId	Subtree expansion; landings use slugs
Location	region, district, town, community	IDs internally; cascade in the panel
Community refinement	community	Optional level-4 narrowing
Verification	verification	VERIFIED / PENDING / UNVERIFIED
Availability	availability	Real field, not inferred
Minimum rating	minRating	Requires at least 3 reviews to trust

Filter	Parameter	Notes
Maximum starting price	maxPrice	Integer pesewas bound
Provider type	providerType	INDIVIDUAL or BUSINESS
Distance	maxDistanceKm	Only meaningful where coordinates exist

Table 7.1 - Filter surface (all server-side)

7.2 Sorting without false ranking

Order	Behaviour when data is missing
Recommended	The weighted matching score; neutral fill keeps empty profiles mid-pack
Nearest	Haversine distance ascending; providers without distance fall to the end
Highest rated	Rated providers first (3 or more reviews), then average descending
Most experienced	yearsExperience descending
Lowest starting price	Price ascending; quote-required providers sort last
Fastest response	Response rate descending; null rates sort last

Table 7.2 - Sort orders

The rule across every order is that missing information never fabricates a position: a provider without a price is not treated as the cheapest, and a provider without reviews is not treated as the best rated. Each sort is pinned by dedicated checks in the verification suite, including the null-last behaviour and a multi-provider experience ordering case.

7.3 Pagination

Pagination is server-side and reuses the Phase 1 pagination contract: page and pageSize are clamped on the way in, and the response carries page, pageSize, total, totalPages, hasNextPage and hasPreviousPage alongside the items. Nothing ever returns an unlimited provider list, and duplicate suppression is structural - one row per provider profile, with a defensive id-based dedupe pass after scoring - so a provider with five services and three service areas still appears exactly once. Acceptance test 5 pins this: thirteen matching providers return as page one of ten and page two of three with the full meta.

8. Provider Profiles, Public Discovery and Authentication

8.1 The public profile

Each result card and each public profile shows only what actually exists in the database: display name and provider type (individual or the affiliated business), profession, headline, verification status, rating aggregate once reviews exist, years of experience, the real availability field, starting price where published, completed-job count where present, service areas, matched services, portfolio items and published reviews. Nothing is invented to make a card look fuller - an unrated provider reads No reviews yet, an unverified provider reads Unverified, and a provider with no published starting price shows no price.

The profile loader assembles everything in a fixed number of queries and applies privacy gates by construction: reviews expose only the reviewer's first name, the payload carries no email addresses, no phone numbers and no raw coordinates, and suspended or deleted providers return 404 rather than a stale public page. A public profile also renders structured data for search engines, with aggregate ratings included only when real reviews exist.

8.2 Public versus authenticated

Surface	Access
Search, results, filters, sorting	Public
Public provider profile, services, service areas	Public
Public reviews and portfolio	Public
Request Service	Authentication required; intent preserved
Message Provider	Authentication required; intent preserved
Save Provider / Post a Job	Placeholder transitions; real flow is Phase 5

Table 8.1 - Discovery surfaces and their boundaries

8.3 Intent-preserving authentication

Discovery is public by design, but the actions that create obligations require an account. When an anonymous visitor presses Request Service, the interface routes to sign-in with the destination and the intent encoded in the callback URL; after authentication the customer lands back on the provider profile with the intent intact, and the browser E2E verified this round trip with a real sign-in. Because the Phase 5 job-request system does not exist yet, the authenticated button does not pretend to submit: it records the

analytics event, shows an honest notice that job requests arrive with Phase 5, and the architecture is wired so that Provider Profile, Request Service, Job Request and Provider Response connect the moment Phase 5 lands. Message Provider behaves the same way on the existing communication foundation.

9. API Endpoints and Database Changes

9.1 GET /api/discovery/providers

The centralized discovery endpoint is public, rate-limited with the shared search preset, and validated by the same strict schema the service uses - every parameter is enumerated, coerced and bounded, and any unknown key or malformed value is rejected with a 422 validation error rather than silently ignored. Each successful search also writes one analytics event with resolved identifiers and coarse outcome counts, and that write is fire-and-forget: an analytics failure can never fail a search.

Group	Parameters
What	categoryId, categorySlug, serviceId, q
Where	regionId, districtId, townId, communityId, latitude, longitude
Refinement	providerType, verification, availability, minRating, maxPrice, maxDistanceKm
Ordering	sort (recommended, nearest, rating, experience, price, response)
Paging	page (1-based), pageSize (1-100, default 20)

Table 9.1 - Query parameters (all Zod-validated)

The response envelope mirrors the service contract: criteria echo with human labels, the resolved location context with breadcrumb and nearby suggestions, the ranked items, the pagination meta, and the nearby tier present only when nothing matched exactly. A second endpoint, POST /api/discovery/events, accepts the client-side interaction events under the same strict validation.

9.2 Analytics events

The event foundation records the seven behaviours the specification names - provider_search, service_search, location_search, provider_profile_view, filter_used, provider_contact_clicked and request_service_clicked - into a single DiscoveryEvent table whose columns are event type, optional category, service, region, district, town, community and provider identifiers, result count, sort and page. There are no personal-identifier columns, no IP address, no user agent and no free-form payload, and the verification suite asserts exactly that. No dashboard was built; this is a foundation, deliberately.

9.3 Database changes

The phase added exactly one table. The DiscoveryEvent model backs the analytics foundation and nothing else; it arrives with its own migration, 20260912091952_phase4_discovery_events, and introduces no duplicate of any existing provider, service-area, location or category model - the Phase 2 domain is reused as-is, bringing the total to 42 models. The clean-database test in chapter 11 replays all three migrations from scratch and seeds the reference and demo data on the fresh schema.

10. SEO, Performance and Security

10.1 Search-engine visibility

Discovery is the platform's largest organic acquisition surface, so the phase ships database-driven, indexable landing routes: `/find/{category}` and `/find/{category}/{location}` - for example `/find/plumbing/nsawam` - each rendered from live data with a real title, description and heading structure, provider results when they exist, related services and nearby locations. The sitemap currently exposes 50 content-backed URLs, generated only where real category and location data exists, and the dynamic robots policy complements it; parameter-heavy search URLs are not sitemapped, thin pages are not generated, and nothing is indexed that would render empty.

Search URLs are shareable by construction: criteria live in the query string, refreshing preserves the search, no sensitive information is ever placed in a URL, and the verification suite checks that landing pages render with content, titles and result data, that the sitemap lists genuine discovery URLs, and that discovery surfaces respond with the standard security headers and request identifiers.

10.2 Performance posture

The discovery path is one capped, minimal-select query plus a bounded scoring loop, with relation data loaded through filtered Prisma joins rather than per-row queries; there is no N+1 in the hot path and no unbounded take. Existing indexes from Phase 2 serve the filters, and no new index was added without justification - the one new table carries only the indexes it needs. Ranking is pure computation over at most 600 candidates, pagination slices after ranking, and the nearby fallback runs only when the exact search is empty, so the common path pays nothing for it. Caching was deliberately not introduced this phase; the upgrade path in DISCOVERY.md records the conditions (privacy-safe keys, correct invalidation) under which it may be.

10.3 Security and privacy

Discovery inherits the Phase 1 handler pipeline end to end: authentication declared per route (public here), Zod validation before any database call, rate limiting per preset, safe Prisma-only queries, and the standard error envelope that never leaks internals. The privacy rules are absolute: results and profiles carry no passwords, no private audit records, no email addresses, no phone numbers and no raw

coordinates, and the verification suite scans live HTTP payloads to assert it. Strict validation doubles as injection defence - SQL-like junk in any parameter is a 422, not a query - and the ownership guards from Phase 3 continue to protect the provider self-edit surfaces that sit beside discovery, including the rule that no provider can set its own verification status.

11. Verification Evidence

11.1 The automated battery

Suite	Checks	Result
bun run lint	-	0 problems
tsc --noEmit	-	0 errors
bun run verify (Phase 1)	48	48 passed, 0 failed
bun run verify:phase2	106	106 passed, 0 failed
bun run verify:phase3	73	73 passed, 0 failed
bun run verify:phase4	56	56 passed, 0 failed
Total	283	283 passed, 0 failed

Table 11.1 - Full battery, re-executed this session on a clean environment

The Phase 4 suite runs in twelve sections on an isolated database rebuilt from migrations, then repeats its contract checks against the live HTTP server: ranking configuration, the critical acceptance block, location hierarchy, server-side filters, sorting, pagination and duplicates, empty results and the nearby fallback, public profiles, ownership boundaries, analytics, and the live HTTP section covering envelope shape, private-data scans, 422 validation, rate limiting, SEO pages and the sitemap.

11.2 The five critical acceptance tests

#	Scenario	Expected	Result
1	Kwame Plumbing serves Nsawam; search Plumbing + Nsawam	Kwame appears	PASS
2	Accra Plumbing serves only Accra; search Plumbing + Nsawam	Absent from exact results	PASS
3	Nsawam electrician; search Plumbing + Nsawam	Absent	PASS
4	Suspended Nsawam plumber; search Plumbing + Nsawam	Absent as a public result	PASS
5	13 matching providers; search Plumbing + Nsawam	Paginated 10 + 3, never unlimited	PASS

Table 11.2 - Critical acceptance results (fixtures on the isolated database)

Two supporting assertions travel with the block: every exact result genuinely serves Nsawam through a real service-area or primary-location row, and the reason strings rendered on cards are honest and score-free. The same suite also pins the nearby tier: the Electrical plus Akropong case surfaces the Nsawam electrician only under the clearly-labelled nearby block, and a verified-only nearby search keeps the verification hard filter.

11.3 Clean-database rebuild

The clean-database test was executed again this session: the development database was deleted, all three migrations were applied with migrate deploy (the Phase 2 domain, the Phase 3 delivery flag, and the Phase 4 discovery events table), the idempotent seed repopulated reference and demo data - 54 towns, 4 provider profiles, 41 categories - and the server came up healthy against the fresh schema. The Nsawam landing page rendered Kwame Darko with his real service areas, and the Phase 3 and Phase 4 suites were re-run in full against the rebuilt environment: 73 and 56 checks, all passing.

11.4 Browser end-to-end

The browser pass walked the complete customer journey in 21 steps with zero console errors and zero page errors: homepage search panel with the database combobox and cascade; the Find navigation; selecting Plumbing, Eastern, Nsawam Municipal and Nsawam; the results page announcing one provider found serving Nsawam with the criteria in a shareable URL; the provider profile showing services, service areas (Nsawam, Adoagyiri, Suhum, Koforidua), an honest Unverified status and no fabricated reviews; the filters and sort controls persisting to the URL; Request Service redirecting anonymous visitors to sign-in with the intent preserved, the sign-in round trip returning the customer to the profile, and the authenticated press producing the honest Phase 5 notice rather than a fake submission; a verified-only filter producing the honest empty state with nearby-area suggestions and real distances; and the nearby tier demonstrated live with the Ejisu search.

Viewport	Width	Horizontal overflow	Console errors
Mobile	390 px	None	Zero
Tablet	768 px	None	Zero
Desktop	1440 px	None	Zero

Table 11.3 - Device matrix (find results, provider profile, homepage)

One accessibility detail was investigated during the pass: an accessibility-tree snapshot appeared to show a missing space inside the results heading. The DOM text content was inspected directly and reads Plumbing in Nsawam correctly - the artifact is a snapshot-serializer quirk, not a rendering defect, and no code change was warranted.

12. Operational Findings, Known Issues and Deferrals

12.1 Operational findings from this session

The working session began in a rebuilt sandbox, which resurrected two known operational behaviours. First, the untracked .env had lost its AUTH_SECRET and the development database was empty - the same environment-rebuild signature recorded in the Phase 3 report. The secret was regenerated into .env, the idempotent seed re-ran, and the server restarted; verify:phase3 then passed 73 of 73. This remains an environment artefact, not a code defect, but it is worth restating that a fresh clone needs DATABASE_URL, AUTH_SECRET and a seed before any authenticated or discovery check can pass. Second, 179 tracked files showed mode-only permission diffs (0644 to 0755) from the rebuild; the recorded modes were restored and the tree verified clean with zero content changes.

Two smaller findings were handled during verification: the background dev server needed a detached start to survive between tool sessions, and the accessibility-snapshot heading quirk described in chapter 11 was disproved against the DOM. None of these findings affected the shipped code; all are recorded here because the phase standard is that every claim is backed by evidence, including the uneventful ones.

12.2 Known issues and limits

- Candidate cap: ranking is computed over at most 600 eligible providers per search, a documented central constant; beyond Ghana-scale data the ranked pagination should move into PostgreSQL.
- Free-text search uses SQLite contains matching; the PostgreSQL upgrade path (citext or trigram indexes) is documented in DISCOVERY.md and Phase 2 notes.

- The nearby tier requires a real search point (requested town or explicit coordinates) to measure distance; without coordinates the interface offers hierarchy-based nearby areas instead of distance-ranked ones.
- Ranking weights are product defaults; they are tuned centrally and should be revisited once real engagement data exists.
- Availability is the provider's declared field, not a live presence signal; the interface words it accordingly and Phase 5 job responses will provide the real signal.

12.3 Deferred to later phases

The following are deliberately unbuilt, each behind a clean integration point: the job request workflow and its provider response loop, quotations, payments and checkout, full messaging conversations, WhatsApp and SMS delivery, save-provider favourites, equipment rental transactions, project management, an AI recommendation engine, a construction cost estimator, and any analytics dashboarding beyond the event foundation. The discovery surfaces already speak the language of the next phase - intent-preserving buttons, honest placeholder notices and analytics events on every entry point - so Phase 5 extends the journey rather than re-plumbing it.

13. Git Commits and Phase 5 Recommendation

13.1 Commits and documentation

Commit	Content
8e26c98	Discovery engine, central ranking configuration and discovery API
c3f713d	Discovery-first frontend: homepage search, /find results, SEO landings, provider profiles
84e1213	Phase 4 verification suite (56 checks) and the neutral-fill scoring fix
4943add	Browser E2E findings: hydration-safe intent links, accessibility and copy polish
466c396	Phase 4 work log documentation

Table 13.1 - Phase 4 commits on main (pushed to origin)

Documentation shipped with the code: DISCOVERY.md in fifteen sections covering the architecture, criteria, matching, eligibility, ranking, sorting, pagination, nearby fallback, public-versus-authenticated boundaries, SEO, API, performance, security and the future upgrade path; ARCHITECTURE.md, DATABASE.md and README.md updated for the 42-model schema, the discovery services and the

new verification suite. At the end of the session the working tree was clean, local main matched origin/main, and .env remained untracked with only .env.example versioned.

13.2 Phase 5 recommendation

The recommended Phase 5 scope is Request Service / Job Requests: the job request model and state machine already exist from Phase 2, the discovery surfaces already hand off with preserved intent, and the analytics foundation already records every request-service click - the phase is the natural next increment. Its core journey is specified and staged: find, select provider, describe the job with location and photos, submit, and let the provider respond toward quotation. Messaging should ride the existing conversation models, and the quotation flow should reuse the money policy and Quote models already in place.

Per the standing protocol, Phase 5 has not been started. Phase 4 is complete against every gate condition in the specification - nationwide discovery, the Ghana hierarchy, service and category search, service-area matching, exact and nearby tiers, ranking, filters, sorting, pagination, profiles, public discovery, authentication boundaries, security, the Nsawam acceptance tests, TypeScript, ESLint, all four verification suites, the browser E2E across mobile, tablet and desktop, documentation and a clean git tree - and the project now waits for the project owner's explicit approval before Phase 5 begins.